

Class 07: Routing & Navigation

2110507 Software Development Practice II

Machigar Ongtang, Ph.D.

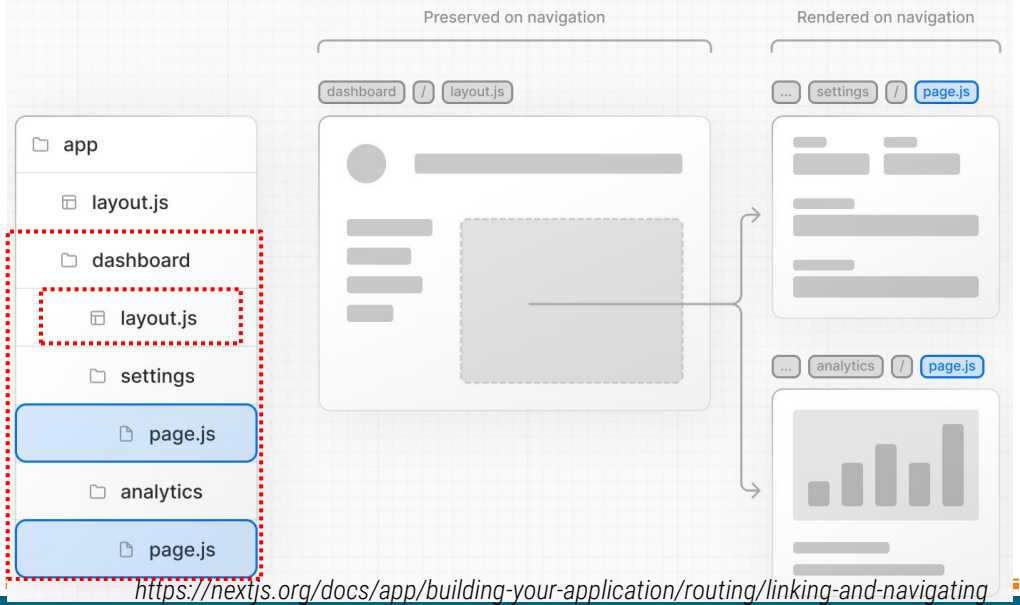
Routing & Navigating



- การ navigate ระหว่าง route ใน Next.js สามารถทำได้ผ่าน 2 รูปแบบ
  - Navigate ผ่าน `<Link>`
  - Navigate ผ่าน `useRouter` Hook
- App Router ใน Next.js อาศัยการทำงานทั้งบน Server และบน Client
  - บน server จะมีการแบ่ง application code โดยอัตโนมัติ ตาม route segments
  - บน client นั้น Next.js ทำการ prefetch (ดึงเนื้อหาล่วงหน้า) และทำการ cache ข้อมูลบน route segment ต่าง ๆ ดังนั้น เมื่อผู้ใช้ navigate ไปยัง route ใหม่ Browser ไม่ต้อง reload หน้า page และเฉพาะ route segment ที่เปลี่ยนจะถูก re-render เป็นการเพิ่มประสิทธิภาพ และเพิ่มระดับ user experience
- Prefetching เป็นการดึงเนื้อหาล่วงหน้า เพื่อ preload route โดยทำงานใน background ก่อนที่ user จะเข้า visit route นั้น ๆ
  - Navigate ผ่าน `<Link>` จะ prefetch route อัตโนมัติ เมื่อ link นั้น ๆ เข้ามาแสดงใน viewport โดยทำเมื่อโหลดหน้า หรือเมื่อเลื่อนหน้ามาจน link เข้ามาแสดงใน viewport
  - Navigate ผ่าน `useRouter` Hook สามารถทำการ prefetch ได้ผ่าน `router.prefetch()`

Routing & Navigating

- **Caching** เป็นการทำ Router Cache หรือ in-memory cache บน client-side โดย Next.js
  - ขณะที่ user ทำการ navigate ภายใน app นั้น จะมีการ cache เก็บ React Server Component Payload ของ route segment ที่ได้รับการ prefetch มา รวมถึงของ route ที่ผ่านการ visit ไปแล้วด้วย
  - Next.js ใช้ข้อมูลจาก cache ให้มากที่สุด แทนที่การดึงข้อมูลใหม่จาก server เพื่อเพิ่มประสิทธิภาพ และลดการ transfer ข้อมูล
- **Partial Rendering** เฉพาะ route segment ที่เปลี่ยนระหว่างการ navigation ที่จะถูก re-render บน client โดย segment ที่แชร์กันช่วง navigation จะถูกคงรักษาไว้



https://nextjs.org/docs/app/building-your-application/routing/linking-and-navigating

Routing & Navigating



- **Soft Navigation** เป็นกลไกใน Next.js ที่ App Router ใช้ soft navigation โดยจะ render เฉพาะ segments ที่เปลี่ยนไป โดยยังคง**รักษา React state และ browser state** เอาไว้ หลีกเลี่ยงการทำ full page reload
  - โดยปกติ Browser ทำ hard navigation ระหว่าง page โดย reload หน้า page และ reset React state (useState hook) และ browser state เช่น ตำแหน่งการ scroll ของ user หรือ element ที่ได้รับ focus

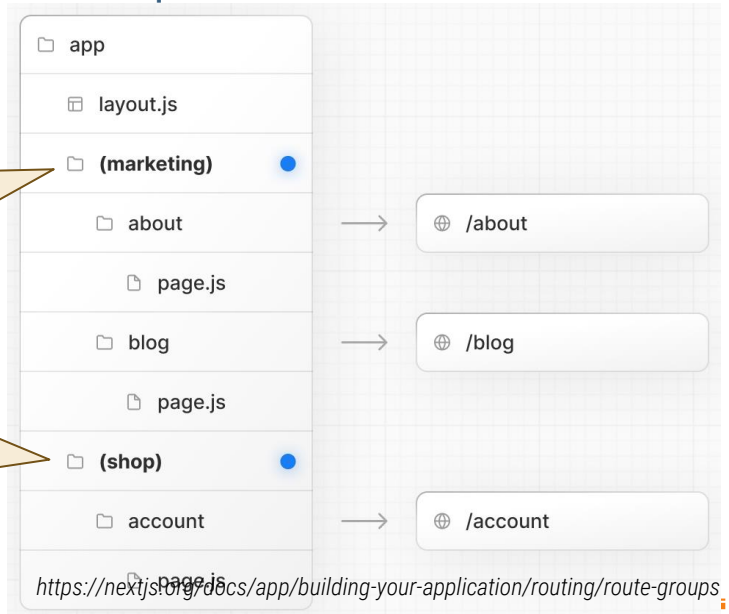
- **Back & Forward Navigation** โดย Next.js เก็บตำแหน่งการ scroll ไว้ใน Router Cache
  - สำหรับ backwards & forwards navigation และการ re-use route segment ต่าง ๆ

Route Groups

- โดย default แล้ว Next.js มอง app directory เป็น root ของ route และ sub-directory ใน app จะถูก map ไปเป็น URL paths
- หากต้องการสร้าง sub-directory เพียงเพื่อใช้ในการบริหารจัดการ route segments หรือจัด project file ให้เป็นกลุ่ม โดยไม่ต้องการให้กระทบกับ URL paths ต้องระบุให้ sub-directory นั้นเป็น Route Group โดยตั้งชื่อไว้ภายใน ( )
  - sub-directory ที่ถูกระบุว่าเป็น Route Group จะไม่ถูกรวมเข้าไปใน URL paths
  - ระวังไม่ให้ route group ต่างกันสร้างเป็น route เดียวกัน เช่น `(marketing)/blog/page.tsx` และ `(shop)/blog/page.tsx` มิฉะนั้นจะมี error

`(marketing)` ใช้ในการจัดกลุ่ม route ที่เกี่ยวข้องกันไว้ด้วยกัน โดย folder นี้จะไม่รวมอยู่ใน URL Path

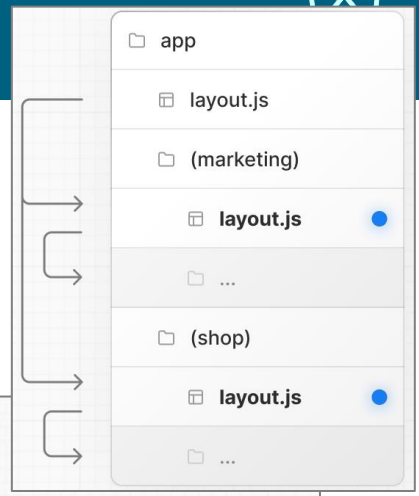
`(shop)` ใช้ในการจัดกลุ่ม route ที่เกี่ยวข้องกันไว้ด้วยกัน โดย folder นี้จะไม่รวมอยู่ใน URL Path



https://nextjs.org/docs/app/building-your-application/routing/route-groups

Route Groups

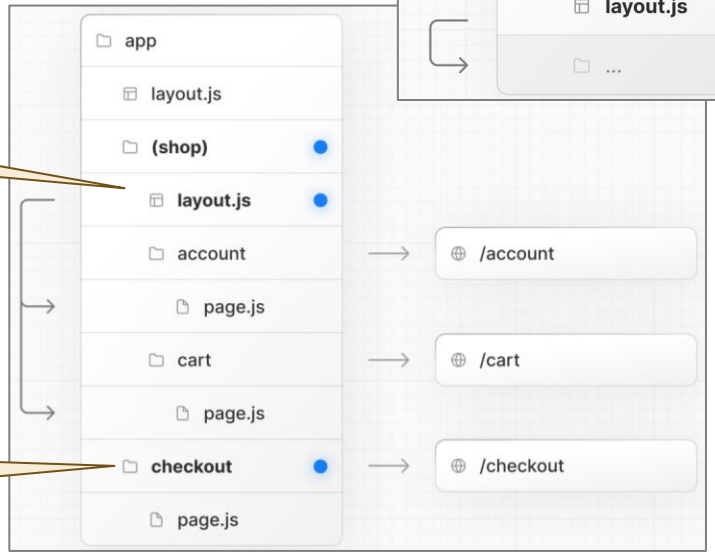
- **Route Group** เอื้อให้ route ต่าง ๆ แม้ว่าจะอยู่ใน URL ลำดับชั้นเดียวกัน แต่ใช้ layout ต่างกันได้
  - แยก route ให้อยู่คนละ route group
  - สร้าง layout ให้ route group โดยการสร้างไฟล์ `layout.js` หรือ `layout.tsx` ไว้ใน group



`/account` และ `/cart` ใช้ layout ร่วมกัน

- เลือกบาง route ให้ใช้ layout ร่วมกัน ขณะที่บาง route ไม่แชร์ layout ร่วมกันได้ โดยแยก route ที่ไม่ต้องการแชร์ layout ด้วยออกมาจาก group

`/checkout` ไม่ใช้ layout ด้วย



Route Groups

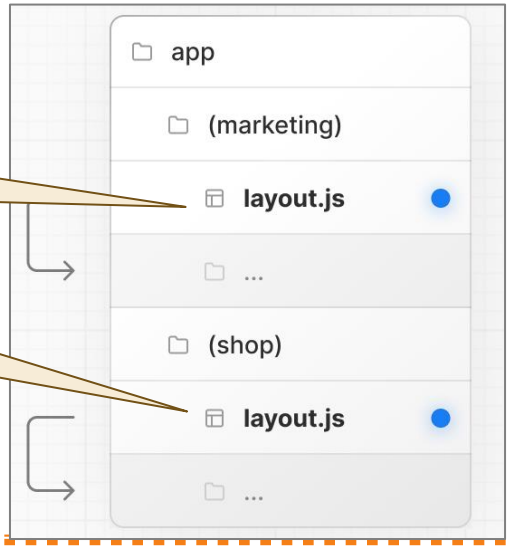
- Route Group เอื้อให้ app สามารถมี Root Layout หลายอันได้
  - เอื้อสำหรับการแบ่ง web application ออกเป็นหลาย section ที่มี UI หลักต่างกัน

ทำได้โดย

- ลบ ไฟล์ `layout.js` หรือ `layout.tsx` ออกจาก `app` directory
- สร้าง route group ที่มี Root Layout ไฟล์ `layout.js` หรือ `layout.tsx` ในแต่ละ route group
- Root Layout ในแต่ละ Route Group ต้องมี `<html>...</html>` tag และ `<body>...</body>` tag ด้วย
- สร้างหน้า homepage ของเว็บ ไฟล์ `page.js` หรือ `page.tsx` ไว้ใน route group ใด route group หนึ่ง

`(marketing)` มี Root Layout ของตนเอง

`(shop)` มี Root Layout ของตนเอง



Dynamic Routes



- **Dynamic Routes** สร้าง route จาก dynamic data โดยที่ไม่ทราบชื่อ route segment ล่วงหน้า
  - สร้าง Dynamic Segment โดยสร้างเป็น folder ที่มีชื่อคร่อมด้วย [] เช่น [id] หรือ [slug]

    ตัวอย่าง สร้าง route app/category/[slug]/page.js
  - ส่ง Dynamic Segments มาในรูปของ params prop มายัง layout, page หรือ route เป็นต้น
    - สำหรับ NextJS 15 การส่ง params ให้กับ route handler เช่น page หรือ layout จะไม่ส่งเป็น plain object แต่จะส่งอยู่ในรูปของ Promise โดย page นั้นจะเป็น async function ซึ่งเป็นเครื่องมือใน JavaScript สำหรับจัดการ การทำงานแบบ asynchronous (จะเรียนใน Class 09)

```
export default async function Page({ params }: Promise<{ params: { slug:string }>})
{
     const {slug} = await params
     return <div>Category: {slug}</div>
}
```

  - Dynamic Segment จะถูกเติมในช่วง request time หรือ pre-rendered ตอน build time

    ตัวอย่าง Route app/category/[slug]/page.js

    ค่า Param { slug: 'a' } URL ที่ได้ /category/a

Dynamic Routes



- รองรับการขยายไปยัง segments ต่อ ๆ ไปใน path โดยการเพิ่ม ... ไว้หน้าชื่อ folder
  - ตัวอย่างเช่น `app/category/[...slug]/page.js`
    จะหมายรวมถึง `/category/tech`
    `/category/tech/cloud`
    ฯลฯ
  - Dynamic Segment จะถูกเติมด้วย `params prop`

ตัวอย่าง

Route `app/category/[...slug]/page.js`
ค่า Param `{ slug: ['tech'] }`
URL ที่ได้ `/category/tech`

Route `app/category/[...slug]/page.js`
ค่า Param `{ slug: ['tech','cloud'] }`
URL ที่ได้ `/category/tech/cloud`

Dynamic Routes



- รองรับ segment ใน path อาจเป็น optional ได้ โดยการใช้ `[[ ]]`
  - ตัวอย่างเช่น `app/category/[[...slug]]/page.js`

    จะหมายรวมถึง `/category`

    `/category/tech`

    `/category/tech/cloud`

    ฯลฯ
  - Dynamic Segment จะถูกเติมด้วย `params prop`

    ตัวอย่าง

    Route `app/category/[[...slug]]/page.js`

    ค่า Param `{}`

    URL ที่ได้ `/category`

    Route `app/category/[[...slug]]/page.js`

    ค่า Param `{ slug: ['tech','cloud'] }`

    URL ที่ได้ `/category/tech/cloud`

Dynamic Routes



- Typescript รองรับการกำหนด type ให้กับ params prop
  - สำหรับ NextJS 15 การส่ง params ให้กับ route handler เช่น page หรือ layout จะไม่ส่งเป็น plain object แต่จะส่งอยู่ในรูปของ Promise
  - Promise พร้อมทั้ง async/await เป็นเครื่องมือใน JavaScript สำหรับจัดการ การทำงานแบบ asynchronous ช่วยให้เขียนโค้ดที่ทำงานแบบ asynchronous โดยเขียนในลักษณะเดียวกับการเขียนโค้ดปกติ (จะเรียนใน Class 09)

```
export default async function Page({ params }: { params: Promise<{ category: string, pid: string }> })
{
   const {category, pid} = await params
   return <div>Category: {category} with Product id {pid}</div>
}
```

  - รองรับ Route app/category/[category]/[pid]/page.js

Link



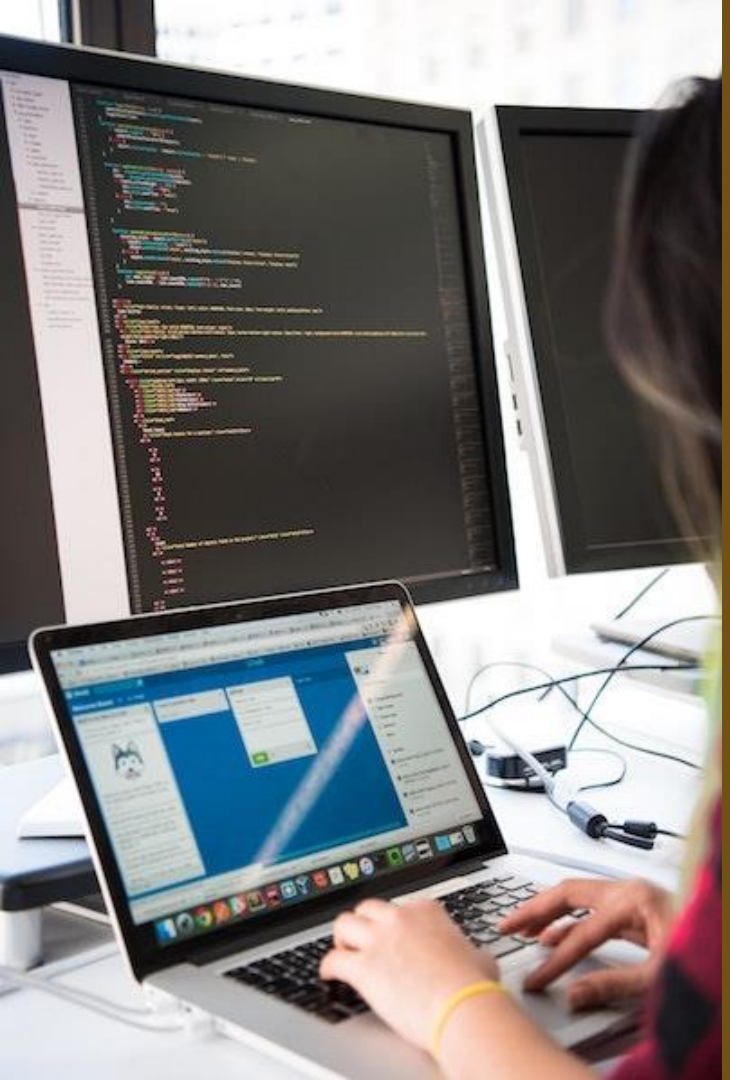
- Navigate ผ่าน `<Link>` component เป็น built-in component ใน Next.js ที่ต่อยอดจาก HTML `<a>` tag เพื่อใช้สำหรับการทำ prefetching และ navigation ไปยัง route ต่าง ๆ บนฝั่ง client-side
  - ใช้งานโดยการ import `Link` Component จาก `next/link`

```
import Link from 'next/link'
```

  - ระบุปลายทางผ่าน `href` prop ตัวอย่างเช่น

```
<Link href="/dashboard">Dashboard</Link>
```

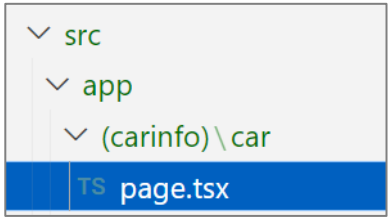
  - ตัวอย่าง props อื่น ๆ
    - `scroll` ถ้า `true` จะ scroll ไปยังด้านบนสุดของ route ใหม่ ถ้า `false` จะรักษา scroll position เดิม
    - `prefetch` ถ้า `true` จะ pre-fetch หน้าของ route ใหม่ เมื่อตัว link ถูก load มาแสดงใน Viewport
    - `replace` ถ้า `true` จะเป็นการแทนที่สถานะปัจจุบันใน history ของ Browser ถ้า `false` จะเพิ่ม URL ใหม่ เข้าไปใน history ของ Browser



Workshop 7.1: Route Group & Dynamic Routes

Route Group

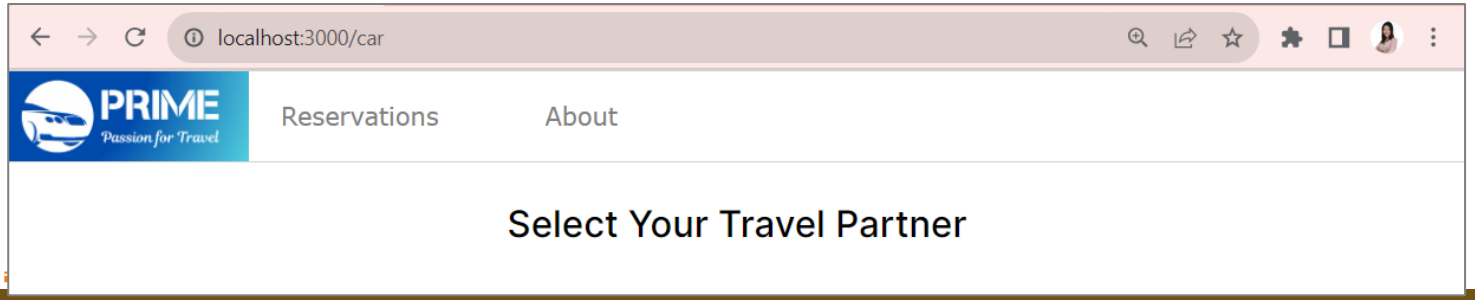
- ทดลองสร้าง Route Group โดยสร้าง folder `(carinfo)` ไว้ใน `app`
  - สร้าง folder `app\(carinfo)\car` และ สร้างหน้าเพจ `page.tsx` สำหรับ route นี้



app > (carinfo) > car > page.tsx > Car

```tsx
export default function Car() {
    return (
        <main className="text-center p-5">
            <h1 className='text-xl font-medium'>Select Your Travel Partner</h1>
        </main>
    )
}
```

  - ลอง Navigate ไปที่ http://localhost:3000/car จะเห็นได้ว่า folder `(carinfo)` ไม่อยู่ใน route segment

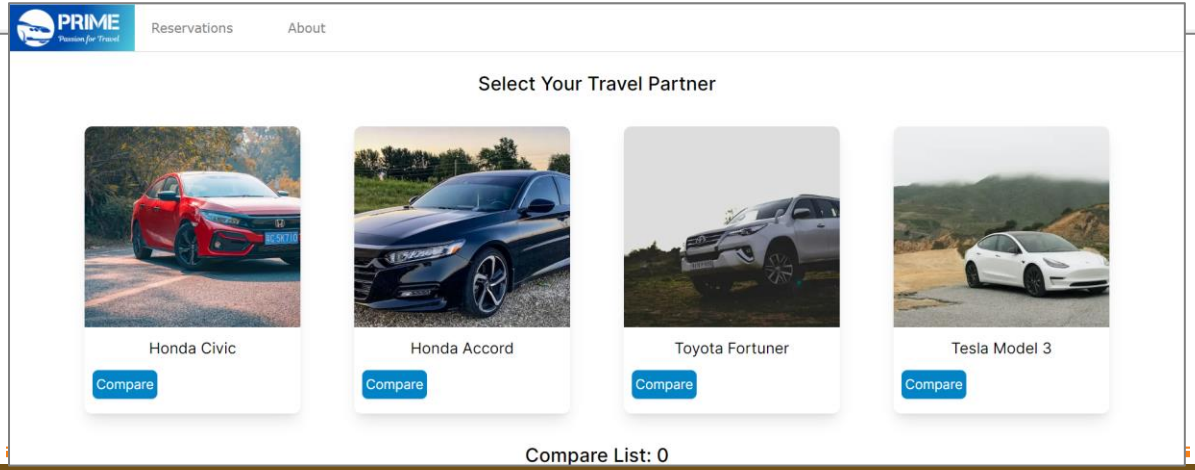


Route Group

- ย้าย CarPanel จากหน้า app\page.tsx มาไว้ใน app\(carinfo)\car\page.tsx

```tsx
export default function Home() {
  return (
    <div className={styles.page}>
      <main className={styles.main}>
        <Banner/>
      </main>
    </div>
  );
}
```

```tsx
import CarPanel from "@/components/CarPanel"
export default function Car() {
    return (
        <main className="text-center p-5">
            <h1 className='text-xl font-medium'>Select Your Travel Partner</h1>
            <CarPanel/>
        </main>
    )
}
```



Link Component & Dynamic Routes



- ใน components\CarPanel.tsx จำลองการดึงข้อมูล
  - Mock ข้อมูลรายการรถใน array ของ Object
  - Loop เพื่อนำข้อมูลรถแต่ละคันมาแสดงบน Card

```
const [ compareList, dispatchCompare ] = useReducer(compareReducer, new Set<st

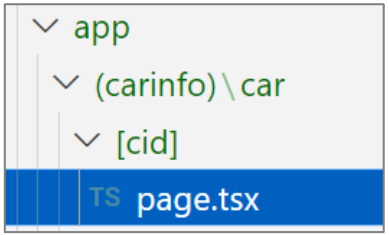
/**
 *  Mock Data for Demonstration Only
 */
const mockCarRepo = [{cid:"001", name:"Honda Civic", image:"/img/civic.jpg"},
        {cid:"002", name:"Honda Accord", image:"/img/accord.jpg"},
        {cid:"003", name:"Toyota Fortuner", image:"/img/fortuner.jpg"},
        {cid:"004", name:"Tesla Model 3", image:"/img/tesla.jpg"}]

return (
```

```
<div style={{margin:"20px", display:"flex",
flexDirection:"row", alignContent:"space-around",
justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
    {
        mockCarRepo.map((carItem)=>(
            <ProductCard carName={carItem.name} imgSrc={carItem.image}
            onCompare={ (car:string)=>dispatchCompare({type:'add', carName:car}) }
            />
            )
        )
    }
</div>
```

Link Component & Dynamic Routes

- สร้าง Dynamic Segment โดยสร้าง folder app\(carinfo)\car\[cid]
  - สร้างหน้า [cid]\page.tsx

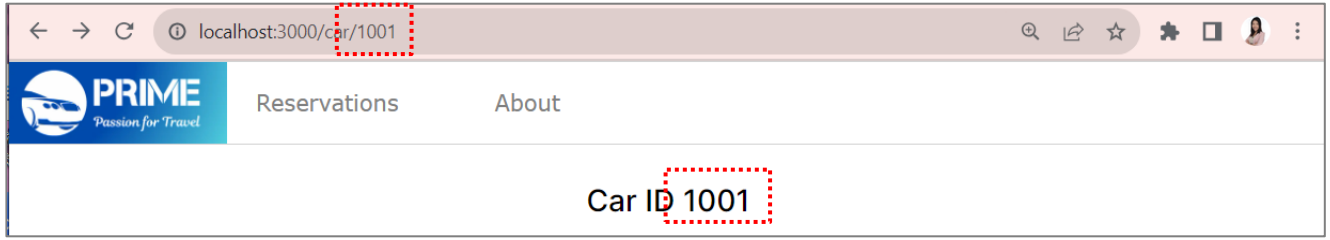



```tsx
export default async function CarDetailPage({params}: {params: Promise<{cid: string}>}) {
    const { cid } = await params
    return (
        <main className="text-center p-5">
            <h1 className='text-lg font-medium'>Car ID {cid}</h1>
        </main>
    )
}
```

รับ params เป็น Promise สำหรับ object ที่มี attribute cid ชนิด string ซึ่งเป็นส่วน dynamic segment

สามารถนำค่า attribute cid มาใช้งานได้

- ลอง Navigate ไปที่ http://localhost:3000/car/1001 จะเห็นได้ว่า dynamic segment ใช้งานได้



Link Component & Dynamic Routes



- ในหน้า `[cid]\page.tsx` จำลองการดึงข้อมูลที่สัมพันธ์กับ `cid`
  - Mock Mock ข้อมูลรถใน Map ของ Object โดยมี `cid` เป็น key
  - ดึงข้อมูลรถที่สัมพันธ์กับ `cid` มาแสดง

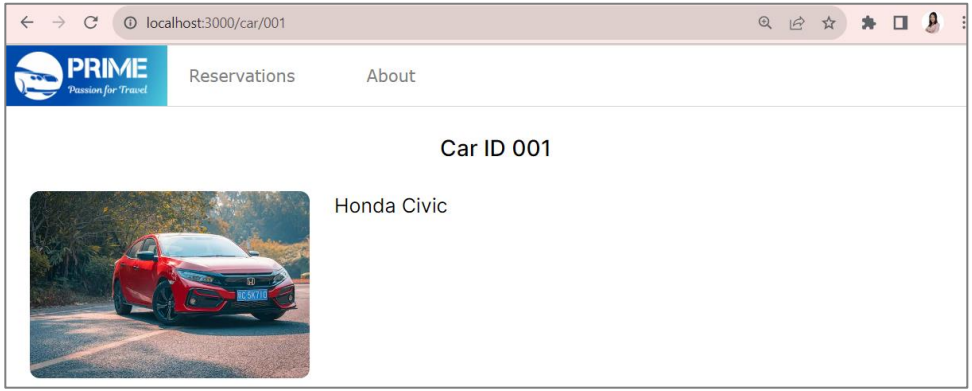
```tsx
export default async function CarDetailPage({params}: {params: Promise<{cid: string}>}) {

    const { cid } = await params
    /**
     *  Mock Data for Demonstration Only
     */
    const mockCarRepo = new Map()
    mockCarRepo.set("001", {name:"Honda Civic", image:"/img/civic.jpg"})
    mockCarRepo.set("002", {name:"Honda Accord", image:"/img/accord.jpg"})
    mockCarRepo.set("003", {name:"Toyota Fortuner", image:"/img/fortuner.jpg"})
    mockCarRepo.set("004", {name:"Tesla Model 3", image:"/img/tesla.jpg"})
```

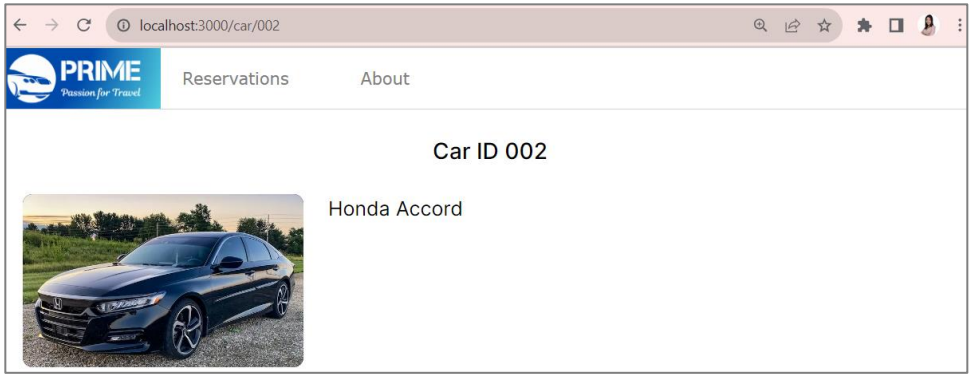
```tsx
<h1 className='text-lg font-medium'>Car ID {cid}</h1>
<div className="flex flex-row my-5">
<Image src={(mockCarRepo.get(cid)).image}
    alt='Product Picture'
    width={0} height={0} sizes="100vw"
    className='rounded-lg w-[30%] bg-black'/>
<div className='text-md mx-5'>{(mockCarRepo.get(cid)).name}</div>
</div>
```

Link Component & Dynamic Routes

- ลอง Navigate ไปที่ http://localhost:3000/car/001



- ลอง Navigate ไปที่ http://localhost:3000/car/002



Link Component & Dynamic Routes



- ใน components\CarPanel.tsx นำ link ไปครอบ ProductCard
  - ใช้ Template Literal ซึ่งคร่อมด้วย ` (backtick) และ $ ในการระบุ variable ใน string ที่เป็นค่าของ href
  - Link นี้กลายเป็น flex item ของ flex container แทน ProductCard จึงกำหนดให้มีความกว้างเป็น 1/5

```
<div style={{margin:"20px", display:"flex",
flexDirection:"row", alignContent:"space-around",
justifyContent:"space-around", flexWrap:"wrap", padding:"10px"}}>
    {
        mockCarRepo.map((carItem)=>(
          <Link href={`/car/${carItem.cid}`} className="w-1/5" key={carItem.cid}>
          <ProductCard carName={carItem.name} imgSrc={carItem.image}
          onCompare={ (car:string)=>dispatchCompare({type:'add', carName:car}) }
          />
          </Link>
        )
    }
</div>
```

Link Component & Dynamic Routes



- ใน components\InteractiveCard.tsx
  - ปรับความกว้างให้เป็น full เนื่องจากจะกลายเป็น child ของ Link component
  - ลบ event handler onClick บน Card

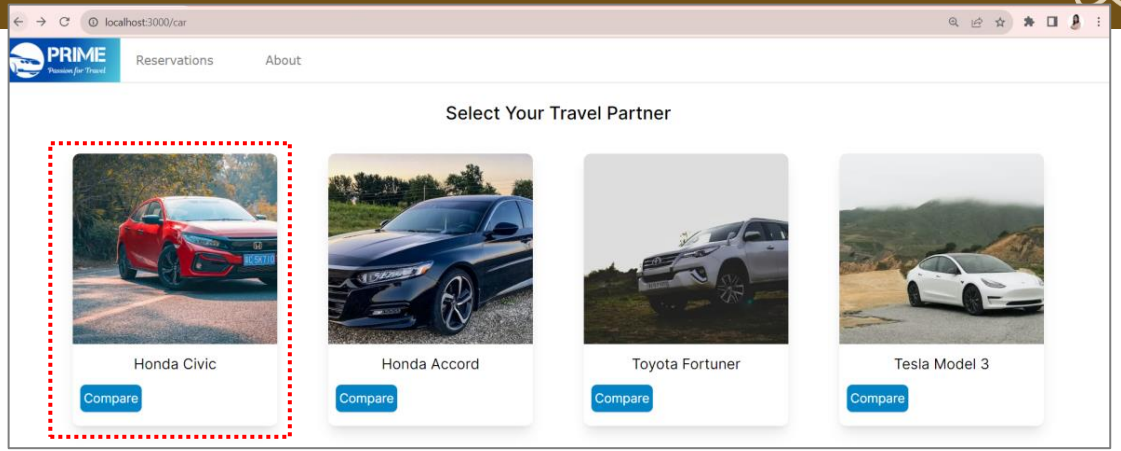
```
return (
    <div className='w-full h-[300px] rounded-lg shadow-lg'
    onMouseOver={ (e)=>onCardMouseAction(e) }
    onMouseOut={ (e)=>onCardMouseAction(e) }>
        { children }
    </div>
);
```

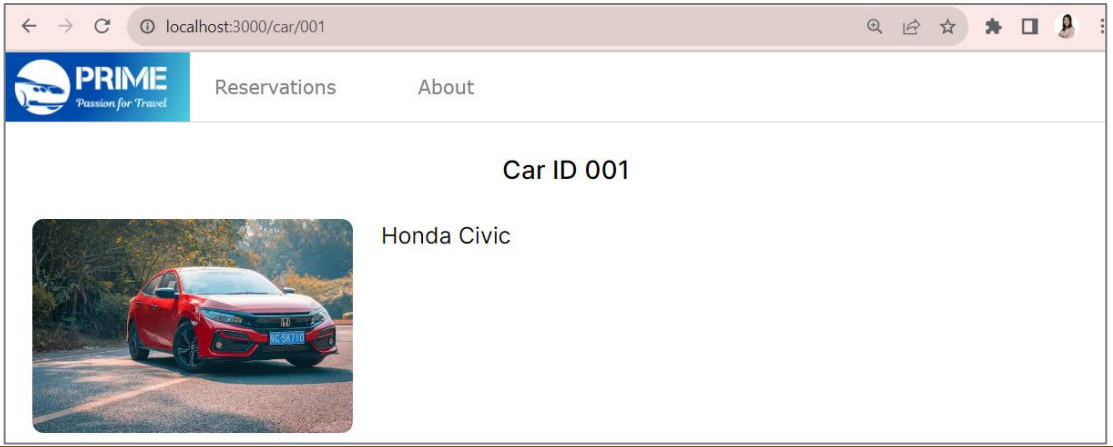
- ใน components\ProductCard.tsx
  - เราไม่ต้องการให้ navigate ตาม link เมื่อกดปุ่ม compare จึงต้องป้องกันไม่ให้ทำ default action ของ Browser ด้วยคำสั่ง *eventObj*.preventDefault()

```
<div className='w-full h-[15%] p-[10px]'>{carName}</div>
<button className='block h-[10%] text-sm rounded-md bg-sky-600
hover:bg-indigo-600 mx-2 px-1 py-1 text-white shadow-sm'
onClick={ (e)=>{ e.preventDefault(); onCompare(carName) } }
>
    Compare</button>
```

Link Component & Dynamic Routes

- ทดสอบการ navigate ไปยัง dynamic segment ผ่าน Link





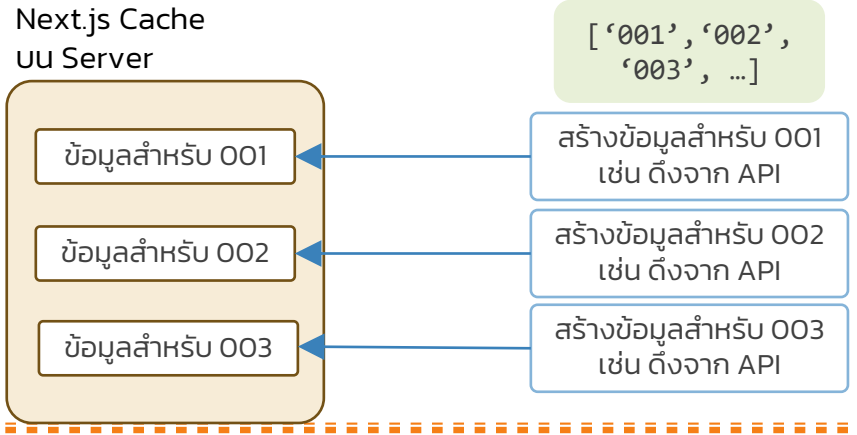
Dynamic Routes with Static Params

- Dynamic Routes โดยปกติไม่ทำ caching เนื่องจาก route เป็นแบบ dynamic
- สำหรับเนื้อหาที่ไม่มีการเปลี่ยนบ่อย เราต้องการทำ caching เพื่อเพิ่มประสิทธิภาพของ application
- Next.js สนับสนุนการทำ Dynamic Routes with Static Params เพื่อทำ caching
  - มี function สำหรับสร้าง array ของค่าที่จะส่งไปให้กับ `params prop` ในช่วง build time

```
export default function Page({ params }: { params: { slug: string } })
```

['001','002', '003', …]

- ช่วง build time จะมีการสร้าง array ของค่าที่จะส่งไปให้กับ `params prop` แล้ว มีการสร้าง content สำหรับ dynamic route แต่ละอันจากชุดของ `params prop` ที่ระบุไว้ แล้ว cache ไว้



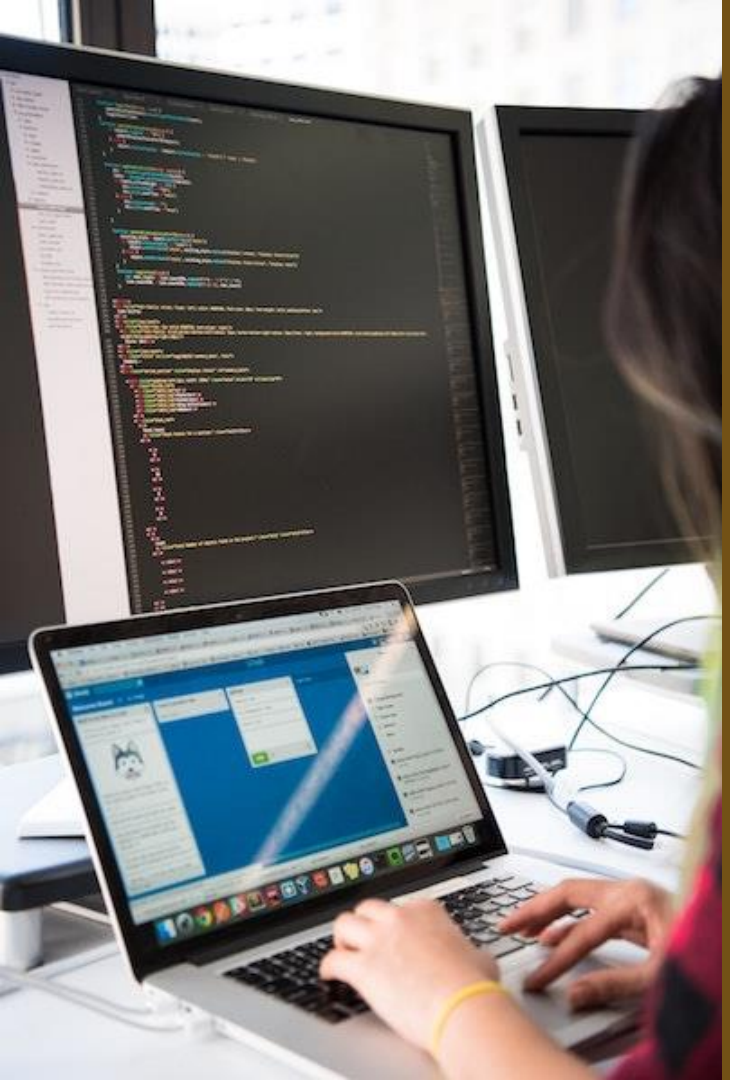
Dynamic Routes with Static Params



- สร้าง Static Params สำหรับ Dynamic Routes เพื่อสร้าง route ที่ถูก generate แบบ static ตอน build time โดยสร้าง generateStaticParams function ไว้ใน page.js หรือ page.tsx

```
export async function generateStaticParams()
{
   // … Fetching data to generate params …
   // return object { slug: value, }
}
```

  - Next.js มีการ cache data ดังนั้น หน้า function ของหลาย page มีการดึงข้อมูลด้วย argument เดียวกับการดึงข้อมูลที่เคยมีการ cache ผลลัพธ์เอาไว้ จะไม่ต้องดึงข้อมูลอีก



Workshop 7.2: Dynamic Routes with Static Params

Dynamic Routes with Static Params



- ในหน้า [cid]\page.tsx ลองสร้าง generateStaticParams function ไว้ใน page.tsx

```
            </main>
        )
}

export async function generateStaticParams() {
    return [{cid:'001'},{cid:'002'}, {cid:'003'}, {cid:'004'}]
}
```

useRouter



- Navigate ผ่าน useRouter Hook รองรับการทำ client-side route transition ระหว่าง page แบบ programmatic
  - ทำงานบน Client Component เท่านั้น โดย import จาก next/navigation

```
import { useRouter } from 'next/navigation'
```

useRouter Hook เป็น React Hook สำหรับเข้าถึง router เพื่อใช้ navigate ด้วย command

```
const router = useRouter()
```

- Push: การ navigate ไปยัง route ที่ระบุ โดยมีการเพิ่มประวัติเข้าไปใน Browser History ใช้คำสั่ง push ตามรูปแบบ

```
router.push(href: string, { scroll: boolean })
```

เช่น

```
const router = useRouter()
router.push('/dashboard')
router.push('/dashboard', { scroll: false })
```

useRouter



- **Replace**: การ navigate ไปยัง route ที่ระบุ โดยไม่เพิ่มประวัติเข้าไปใน Browser History ใช้คำสั่ง replace ตามรูปแบบ

```
router.replace(href: string, { scroll: boolean })
```

- **Refresh** เพื่อ refresh route ปัจจุบันโดยส่ง request ใหม่ไปยัง server เพื่อ re-fetch data request และ re-render Server Components โดยไม่กระทบ client-side React state หรือ browser state (เช่น scroll position)

```
router.refresh()
```

- **Prefetch** สั่ง prefetch route ที่ระบุ เพื่อให้การเปลี่ยน route ฝั่ง client-side เร็วขึ้น

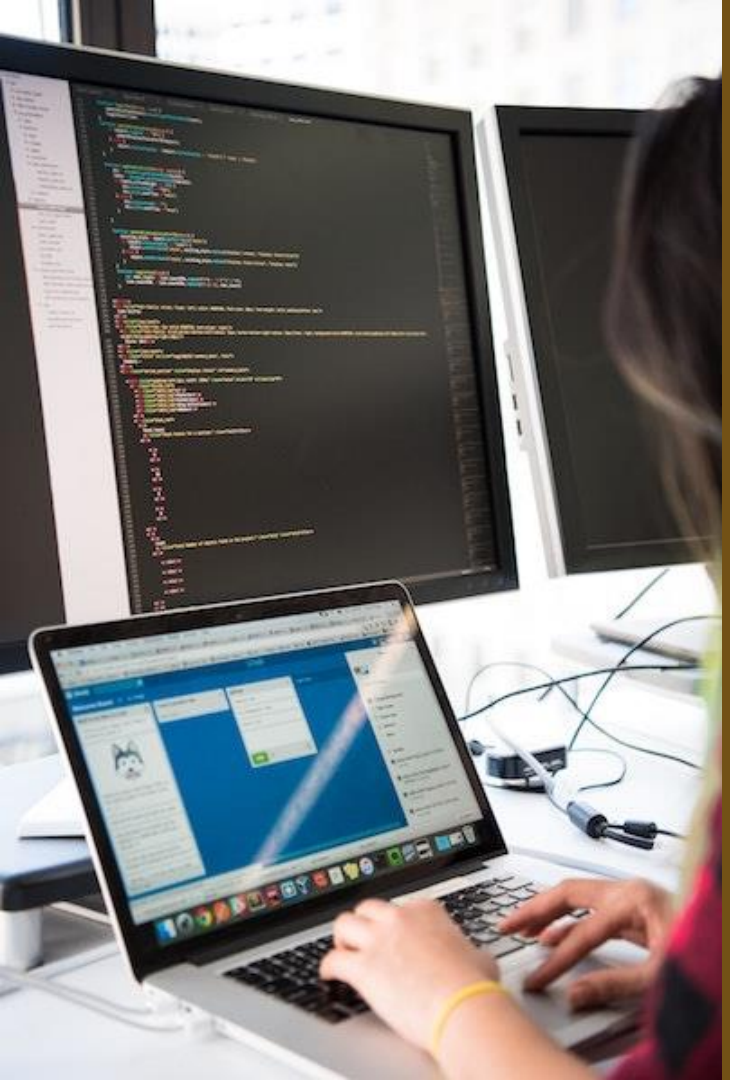
```
router.prefetch(href: string)
```

- **Back** กลับไปยัง route ก่อนหน้า ซึ่งก็คือ route ก่อนหน้าที่อยู่ใน browser history stack

```
router.back()
```

- **Forward** ไปยัง route ถัดไป ซึ่งก็คือ route ถัดไปที่อยู่ใน browser history stack

```
router.forward()
```

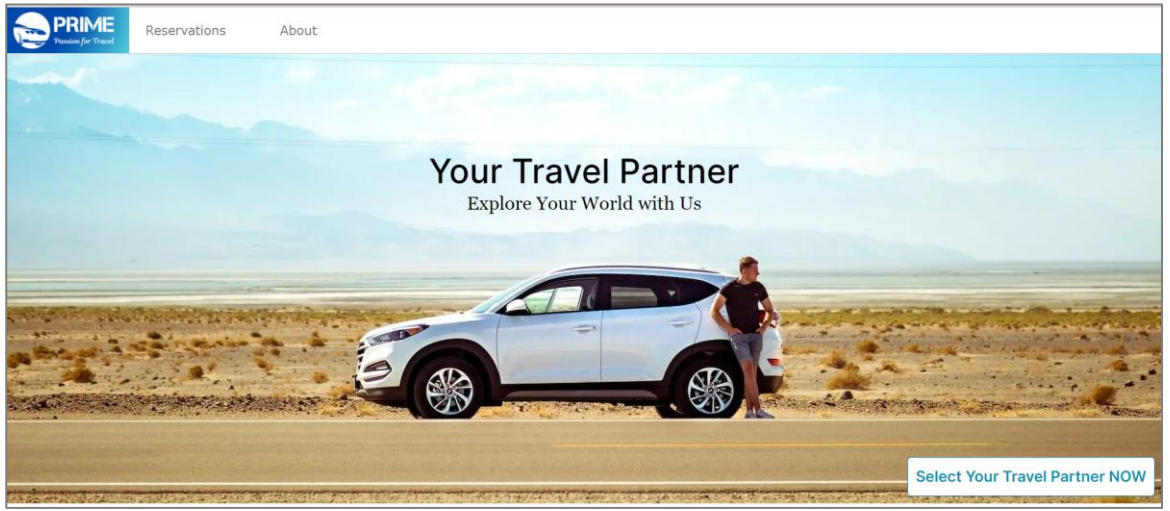


Workshop 7.3: useRouter

useRouter

- ใน components\Banner.tsx
  - วางปุ่มลงไปด้านล่างขวาของรูป Banner

```
<div className={styles.bannerText}>
    <h1 className='text-4xl font-medium'>Your Travel Partner</h1>
    <h3 className='text-xl font-serif'>Explore Your World with Us</h3>
</div>
<button className="bg-white text-cyan-600 border border-cyan-600
font-semibold py-2 px-2 m-6 rounded z-30 absolute bottom-0 right-0
hover:bg-cyan-600 hover:text-white hover:border-transparent">
Select Your Travel Partner NOW
</button>
```



useRouter



- ใน components\Banner.tsx
  - Import และใช้ useRouter()

```
import Image from 'next/image';
import { useRouter } from 'next/navigation'

export default function Banner () {
    const covers = ['/img/cover.jpg', '/img/cover2.jpg', '/img/cover3.jpg']
    const [index, setIndex] = useState(0)
    const router = useRouter()

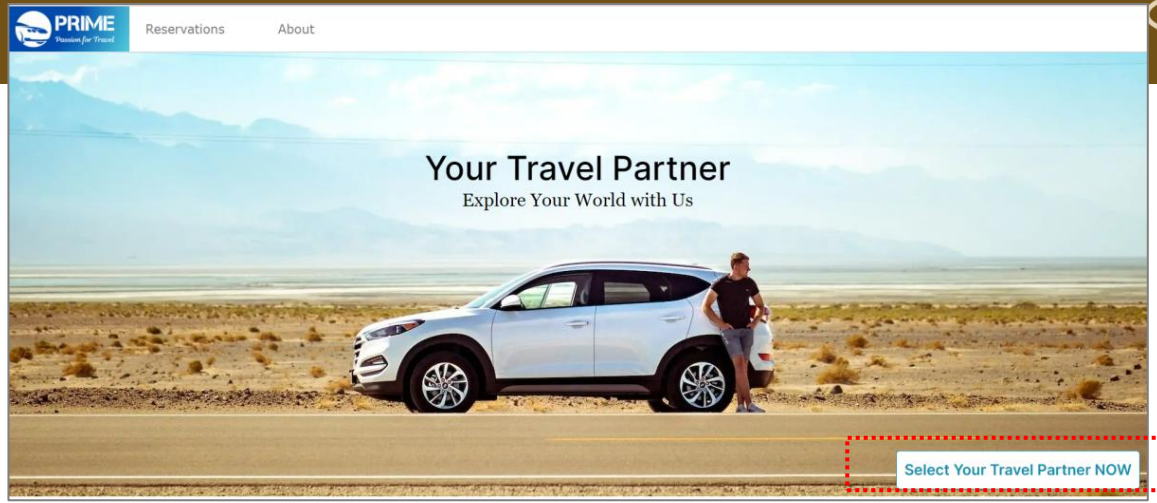
    return (
```

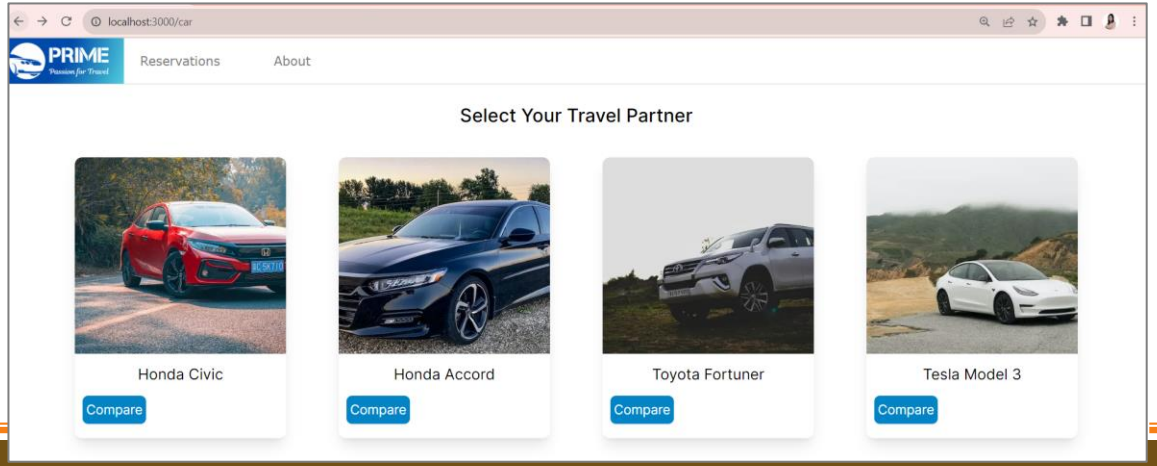
  - เมื่อกดปุ่ม ให้หยุดการ propagate ของ event และใช้ router ในการ navigate ไป route /car

```
<button className="bg-white text-cyan-600 border border-cyan-600
font-semibold py-2 px-2 m-2 rounded z-30 absolute bottom-0 right-0
hover:bg-cyan-600 hover:text-white hover:border-transparent"
onClick={(e)=>{ e.stopPropagation(); router.push('/car') }}>
Select Your Travel Partner NOW
</button>
```

useRouter

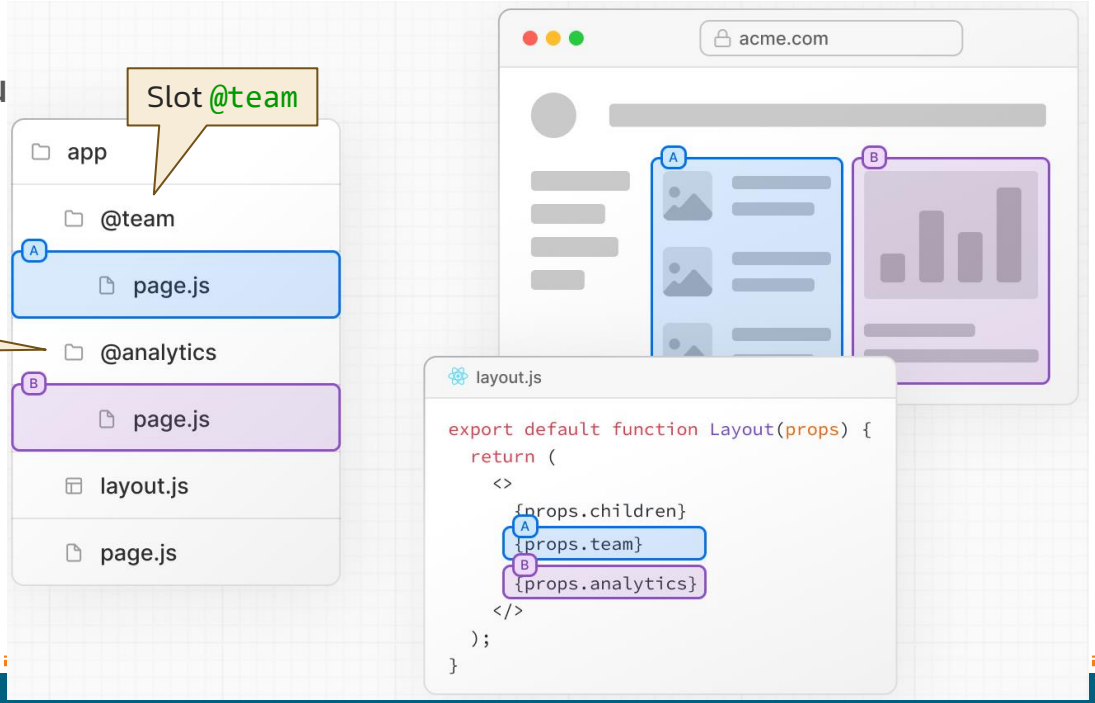
- ทดลองใช้งาน



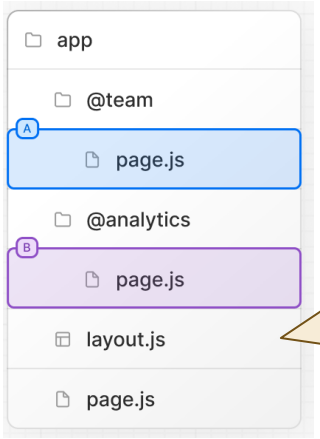


Parallel Routes

- Next.js รองรับ Parallel Routing สำหรับการ render หน้า page ซึ่งอาจมีหลาย page ลงบน layout เดียวกันพร้อม ๆ กัน หรือตามเงื่อนไข
  - เหมาะกับ app ที่มี dynamic section เช่น dashboards หรือ feeds
  - Parallel Routing ใช้ในการสร้าง app ที่มี routing pattern ซับซ้อน
  - สร้าง Parallel Routes ด้วย slot ซึ่งใช้สัญลักษณ์ `@folder`

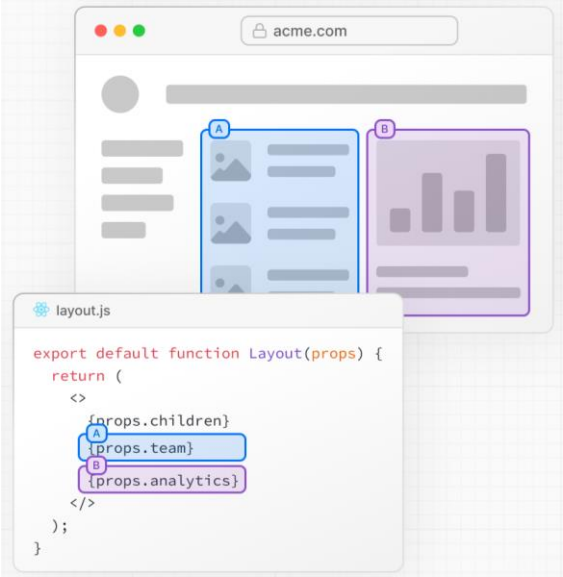


Parallel Routes



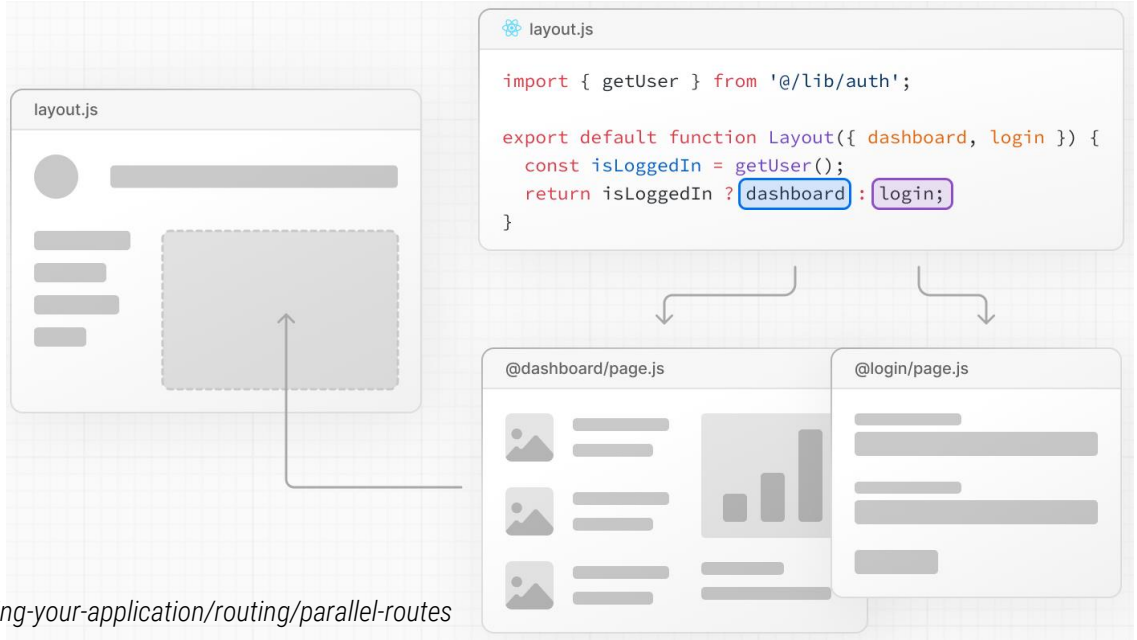
- `layout.js` รับ `@analytics` และ `@team` slots props มาทำการ render ไปพร้อม ๆ กัน พร้อมกับ children prop:

```tsx
export default function Layout(props: {
  children: React.ReactNode
  analytics: React.ReactNode
  team: React.ReactNode
}) {
  return (
    <>
      {props.children}
      {props.team}
      {props.analytics}
    </>
  )
}
```



Parallel Routes

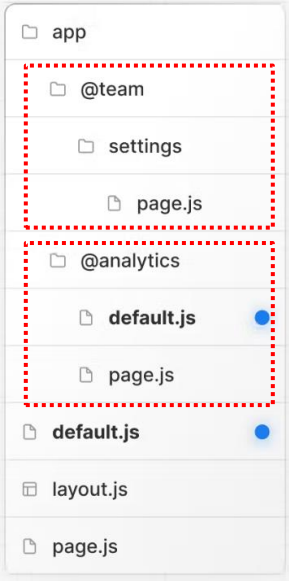
- **Parallel Routing** รองรับการ render slot ตามเงื่อนไข เพื่อแยก code ของ URL เดียวกัน ตามเงื่อนไข

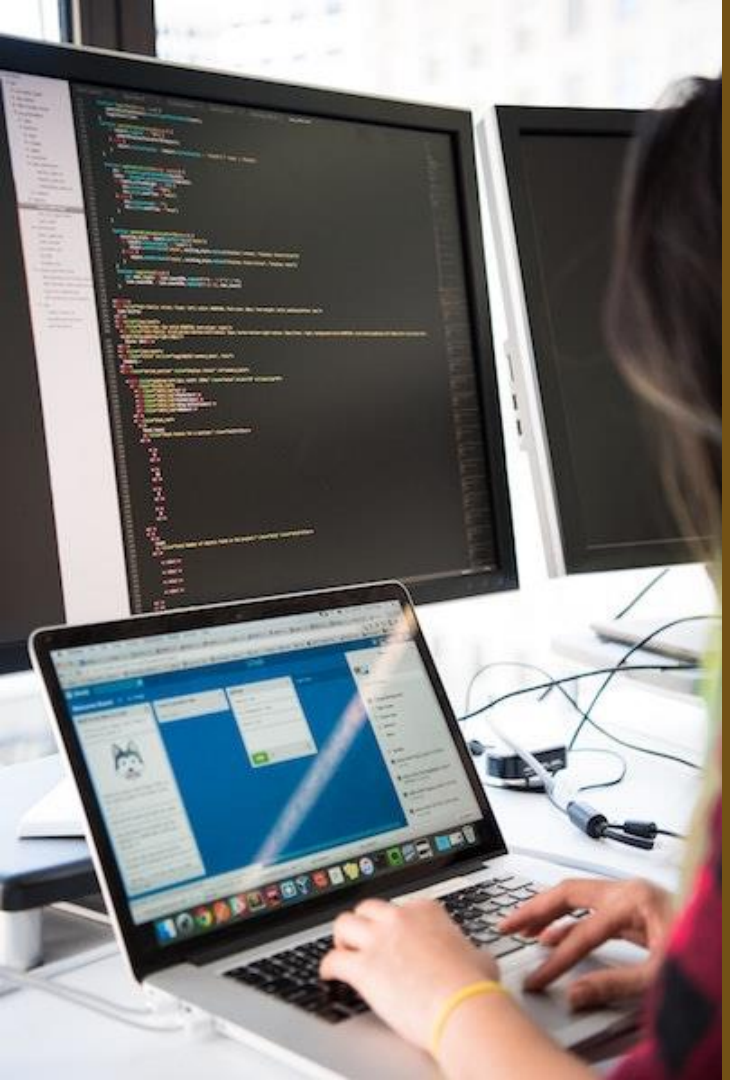


https://nextjs.org/docs/app/building-your-application/routing/parallel-routes

Parallel Routes

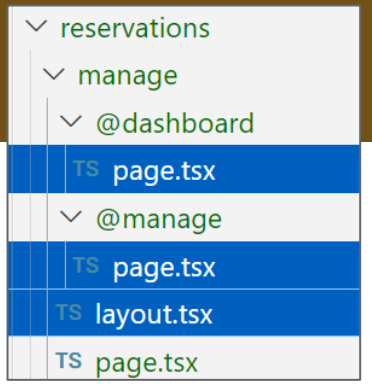
- **Unmatched Routes** คือเมื่อเนื้อหาที่ render ใน slot ไม่ตรงกับ URL ซึ่ง Next.js ต้องรับมือ
  - หาก slot ที่ใช้งานมีโครงสร้าง route ที่ไม่เหมือนกัน ดังเช่นตัวอย่าง
    - Slot `@team` มี `setting` directory แต่ Slot `@analytics` ไม่มี
    - ตอน navigate หากพบ unmatched route จะ render active state ก่อนหน้า
    - ตอน reload หากพบว่ามี unmatched route, Next.js จะ render หน้า `default.js` ของ slot นั้น ๆ แต่ถ้าไม่พบ `default.js` จะแจ้ง error 404





Workshop 7.4: Parallel Routes

Parallel Routes



- ใน `reservations\manage` สร้าง slot `@dashboard` และ `@manage`
  - สร้าง `page.tsx` สำหรับทั้ง slot `@dashboard` และ `@manage`
  - สร้าง `reservations\manage\layout.tsx` สำหรับ route `reservations\manage`
  - หน้า `reservations\manage\@dashboard\page.tsx`

```tsx
export default function DashboardPage () {
    return (
        <main className="bg-slate-100">
            <div>Your Dashboard</div>
        </main>
    );
}
```

  - หน้า `reservations\manage\@manage\page.tsx`

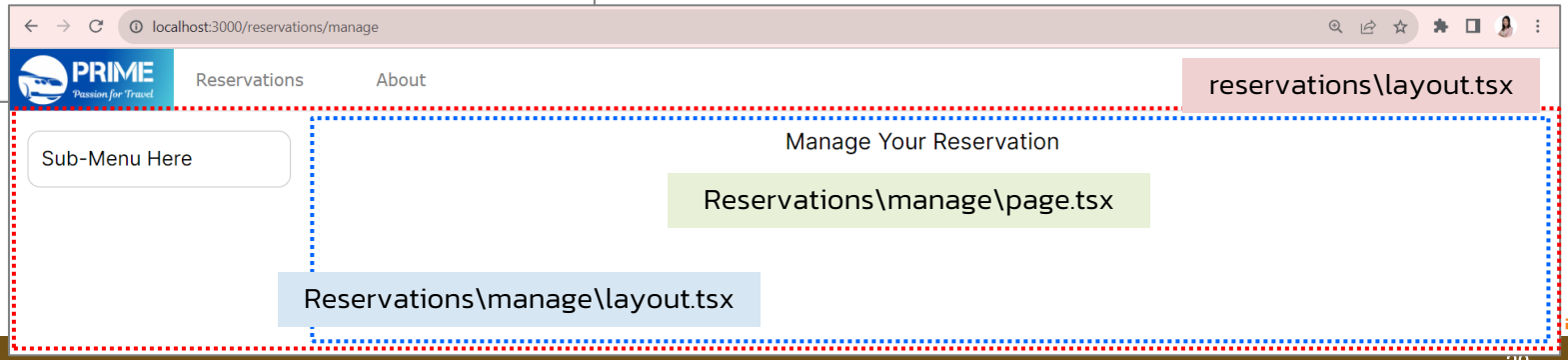
```tsx
export default function ManagePage () {
    return (
        <main>
            <div>Your Reservations</div>
        </main>
    );
}
```

Parallel Routes

- ปรับแต่งหน้า reservations\manage\page.tsx เดิม
- เป็น children สำหรับ reservations\manage\layout.tsx

```tsx
export default function ManageReservations () {
    return (
        <main>
            <div className="text-center text-lg">
                Manage Your Reservation</div>
        </main>
    );
}
```

```tsx
export default function ManageReservationLayout (
    {children}: {children:React.ReactNode} ) {
    return (
        <div className="flex flex-col w-full">
            {children}
        </div>
    );
}
```



Parallel Routes

- ให้ reservations\manage\layout.tsx รองรับ parallel routes จาก slot @dashboard และ @manage

```tsx
export default function ManageReservationLayout ( {children, dashboard, manage}:
    {children:React.ReactNode, dashboard:React.ReactNode, manage:React.ReactNode} ) {
    return (
        <div className="flex flex-col w-full">
            {children}
            {dashboard}
            {manage}
        </div>
    );
}
```

http://localhost:3000/reservations/manage

